

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Analytical Problem Solving
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/nonrestoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	E.Jeevanantham	Reg. No.:	192312406
Section / Batch:	2023	Date:	03 - 08 - 26

Instructions to Students

- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

QUESTIONS

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyse in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floatingpoint representation. Given a decimal number (e.g., -13.25), analyse how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct

I E.Jeevanantham (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

Q1

The 2's complement method is the standard technique used by processors to represent signed binary numbers. Positive numbers are stored in normal binary form, whereas negative numbers are represented by taking the 2's complement of their positive equivalent. In an 8-bit system, the most significant bit (MSB) acts as the sign bit, where 0 represents a positive number and 1 represents a negative number.

For addition, the processor simply adds the two 8-bit numbers using binary addition. Any carry generated beyond the MSB is discarded. For subtraction, the processor converts the subtrahend into its 2's complement and performs binary addition. This allows the same adder circuit to perform both addition and subtraction without requiring separate subtraction hardware.

Overflow occurs only when the result cannot be represented within the available number of bits. In 2's complement arithmetic, this happens when two positive numbers produce a negative result or two negative numbers produce a positive result. The processor detects overflow by comparing the carry into the MSB with the carry out of the MSB or by checking whether the sign of the result differs from the signs of both operands.

Using 2's complement provides several advantages such as a single representation for zero, simpler hardware implementation, faster arithmetic operations, and efficient overflow detection. It is therefore widely used in modern processors and digital systems.

Q2

A Carry Look-Ahead Adder (CLA) is a high-speed adder designed to eliminate the delay caused by carry propagation in a Ripple Carry Adder (RCA). Instead of waiting for each carry to pass through every stage, the CLA predicts the carry values in advance using Generate (G) and Propagate (P) signals. The generate signal indicates whether a stage will produce a carry, while the propagate signal indicates whether an incoming carry will be passed to the next stage.

In a Ripple Carry Adder, each full adder must wait for the carry from the previous stage before producing its output. Therefore, the total delay increases linearly with the number of bits, making it slower for large binary additions. In contrast, a CLA computes all carry signals in parallel, significantly reducing the computation time.

Although a CLA requires more hardware and has a more complex circuit than a Ripple Carry Adder, it provides much faster addition. This makes it the preferred choice in modern processors, ALUs, and high-performance computing systems where speed is more important than hardware complexity.

Q3

Booth's algorithm is an efficient multiplication technique used for signed binary numbers represented in 2's complement form. It uses four registers: the Accumulator (A) to store partial results, the Multiplier (Q), the Multiplicand (M), and an extra bit called the Booth bit (Q_{-1}). At each cycle, the processor examines the pair Q_0Q_{-1} to decide whether to add the multiplicand, subtract it, or perform no operation. After the operation, an arithmetic right shift is performed on the combined registers, preserving the sign bit.

Unlike ordinary binary multiplication, Booth's algorithm efficiently handles both positive and negative numbers without requiring separate sign processing. It also reduces the number of additions and subtractions by treating consecutive 1's in the multiplier as a single operation. This decreases the total number of arithmetic operations, making multiplication faster and more efficient.

Due to its speed and ability to handle signed numbers directly, Booth's algorithm is widely used in processors, arithmetic logic units (ALUs), digital signal processors (DSPs), and embedded systems. It provides better performance than traditional shift-and-add multiplication, especially when the multiplier contains long sequences of 1's.

Q4

Restoring Division is a binary division algorithm in which the divisor is subtracted from the partial remainder after every left shift. If the subtraction produces a negative remainder, the original value is restored by adding the divisor back, and the quotient bit is set to 0. If the remainder is positive, it is retained, and the quotient bit is set to 1. Although this method is simple and easy to implement, the repeated restoration process increases the number of arithmetic operations, making it slower.

Non-Restoring Division improves the efficiency of restoring division by eliminating the immediate restoration step. When the remainder becomes negative, the algorithm proceeds to the next iteration and performs an addition instead of a subtraction. This reduces unnecessary operations and shortens the execution time. Only one final correction may be required if the last remainder is negative.

Comparison

Restoring Division	Non-Restoring Division
Restores negative remainder immediately	Does not restore immediately
More addition/subtraction operations	Fewer arithmetic operations
Slower execution	Faster execution
Simpler hardware	Slightly more complex hardware
Lower efficiency	Higher efficiency

Non-restoring division is preferred in modern processors because it reduces the number of arithmetic operations, improves execution speed, and provides better overall performance while producing the same quotient and remainder as restoring division.

The IEEE 754 standard is the most widely used format for representing floating-point numbers in computers. It stores a number in three fields: Sign, Exponent, and Mantissa (Fraction). The sign bit indicates whether the number is positive or negative, the exponent stores the power of two using a bias value, and the mantissa stores the significant digits of the number.

In single precision, the floating-point number occupies 32 bits, consisting of 1 sign bit, 8 exponent bits, and 23 mantissa bits. In double precision, it occupies 64 bits, consisting of 1 sign bit, 11 exponent bits, and 52 mantissa bits. Double precision provides greater accuracy and a larger range of values than single precision.

During floating-point addition, the exponents of both numbers are first aligned by shifting the mantissa of the smaller exponent. The mantissas are then added or subtracted depending on the signs of the numbers. The result is normalized so that it is in the form $1.xxxxx \times 2^n$. If the mantissa exceeds the available number of bits, it is rounded according to IEEE 754 rounding rules.

Due to the limited number of mantissa bits, precision loss may occur when very large and very small numbers are added together. Rounding errors can also occur because some decimal numbers cannot be represented exactly in binary. Despite these limitations, IEEE 754 provides a standardized, accurate, and efficient method for performing floating-point arithmetic in modern processors.